SOMAIYA VIDYAVIHAR UNIVERSITY
K J Somaiya School of Engineering
(formerly K J Somaiya College of Engineering)

K. J. Somaiya School of Engineering, Mumbai-77

(Formerly K. J. Somaiya College of Engineering)

Department of Electronics Engineering

Course Name:	**Object Oriented Programming**	**Semester:**	**III**
Date of Performance:	___ / ___ / _____	**Batch No:**	**Batch _A4__**
Faculty Name:		**Roll No:**	**16015024082**
Faculty Sign & Date:		**Grade/Marks:**	**___/15**

Writing Program (05)	Performance in lab (05)	Post lab questions, conclusion and completion (02+01+02)

Experiment No: 8
Title: Multithreading in Java

Aim and Objective of the Experiment:
Learn the how to do multithreading in Java

COs to be achieved:
CO4: Develop a Java application with threads and generics classes.

Tools used:

Roll No:_____

K. J. Somaiya School of Engineering, Mumbai-77

(Formerly K. J. Somaiya College of Engineering)

Department of Electronics Engineering

Theory: Multithreading in java, threads, operations with threads, using multiple threads, Thread Class etc.
1. Syntax of implementing thread in Java, thread group, state diagram of thread

Code:
1. Write a Java program that implements a multi-threaded program has three threads. First thread generates a random integer every 1 second and if the value is even, second thread computes the square of the number and prints. If the value is odd the third thread will print the value of cube of the number. (use thread class) 2. Write a Java program which first generates a set of random numbers and then determines negative, positive even, positive odd numbers concurrently. (use runnable interface) 3. Write a Java program to solve the producer-consumer problem, where a producer generates a value and a consumer consumes it before the producer generates the next value.

Roll No:_____

K. J. Somaiya School of Engineering, Mumbai-77

(Formerly K. J. Somaiya College of Engineering)

Department of Electronics Engineering

1.

```java
class NumberGenerator extends Thread {  2 usages
    public void run() {
        Random rand = new Random();
        while (true) {
            int num = rand.nextInt( bound: 100);
            System.out.println("\nGenerated Number: " + num);

            if (num % 2 == 0) {
                new Square(num).start();
            } else {
                new Cube(num).start();
            }

            try {
                Thread.sleep( millis: 1000);
            } catch (InterruptedException e) {
                System.out.println(e);
            }
        }
    }
}

class Square extends Thread {  1 usage
    private int number;  4 usages
    Square(int n) { number = n; }

    public void run() { System.out.println("Square of " + number + " = " + (number * number)); }
}

class Cube extends Thread {  1 usage
    private int number;  5 usages
    Cube(int n) { number = n; }

    public void run() { System.out.println("Cube of " + number + " = " + (number * number * number)); }
}

Rename usages
public class MultiThreadQ1 {
    public static void main(String[] args) {
        NumberGenerator generator = new NumberGenerator();
        generator.start();
    }
```


Roll No:_____

2.

```java
import java.util.*;

class NumberSet {  2 usages  new *
    int[] nums;  6 usages
    NumberSet(int size) {  1 usage  new *
        nums = new int[size];
        Random r = new Random();
        for (int i = 0; i < size; i++) nums[i] = r.nextInt( bound: 201) - 100;
    }
}

class PositiveEven implements Runnable {  1 usage  new *
    int[] arr;  2 usages
    PositiveEven(int[] arr) { this.arr = arr; }  1 usage  new *
    public void run() {  new *
        System.out.print("Positive Even: ");
        for (int n : arr) if (n > 0 && n % 2 == 0) System.out.print(n + " ");
        System.out.println();
    }
}

class PositiveOdd implements Runnable {  1 usage  new *
    int[] arr;  2 usages
    PositiveOdd(int[] arr) { this.arr = arr; }  1 usage  new *
    public void run() {  new *
        System.out.print("Positive Odd: ");
        for (int n : arr) if (n > 0 && n % 2 != 0) System.out.print(n + " ");
        System.out.println();
    }
}

class Negative implements Runnable {  1 usage  new *
    int[] arr;  2 usages
    Negative(int[] arr) { this.arr = arr; }  1 usage  new *
    public void run() {  new *
        System.out.print("Negative: ");
        for (int n : arr) if (n < 0) System.out.print(n + " ");
        System.out.println();
    }
}
```

```java
public class RandomNumberClassifier {  new *
    public static void main(String[] args) {  new *
        NumberSet ns = new NumberSet( size: 15);
        System.out.println("Generated Numbers: " + Arrays.toString(ns.nums));
        Thread t1 = new Thread(new PositiveEven(ns.nums));
        Thread t2 = new Thread(new PositiveOdd(ns.nums));
        Thread t3 = new Thread(new Negative(ns.nums));
        t1.start(); t2.start(); t3.start();
    }
}
```

3.

```java
class SharedBuffer {  6 usages  new *
    private int data;  2 usages
    private boolean available = false;  4 usages

    public synchronized void produce(int value) {  1 usage  new *
        while (available) {
            try { wait(); } catch (InterruptedException e) {}
        }
        data = value;
        available = true;
        System.out.println("Produced: " + value);
        notify();
    }

    public synchronized void consume() {  1 usage  new *
        while (!available) {
            try { wait(); } catch (InterruptedException e) {}
        }
        System.out.println("Consumed: " + data);
        available = false;
        notify();
    }
}

class Producer extends Thread {  1 usage  new *
    SharedBuffer buffer;  2 usages
    Producer(SharedBuffer b) { buffer = b; }  1 usage  new *
    public void run() {  new *
        for (int i = 1; i <= 5; i++) {
            buffer.produce(i);
            try { Thread.sleep( millis: 500); } catch (InterruptedException e) {}
        }
    }
}

class Consumer extends Thread {  1 usage  new *
    SharedBuffer buffer;  2 usages
    Consumer(SharedBuffer b) { buffer = b; }  1 usage  new *
    public void run() {  new *
        for (int i = 1; i <= 5; i++) {
            buffer.consume();
            try { Thread.sleep( millis: 500); } catch (InterruptedException e) {}
```

```
    SharedBuffer buffer;  2 usages
    Consumer(SharedBuffer b) { buffer = b; }  1 usage  new *
    public void run() {  new *
        for (int i = 1; i <= 5; i++) {
            buffer.consume();
            try { Thread.sleep( millis: 500); } catch (InterruptedException e) {}
        }
    }
}

public class ProdCons {  new *
    public static void main(String[] args) {  new *
        SharedBuffer buffer = new SharedBuffer();
        new Producer(buffer).start();
        new Consumer(buffer).start();
    }
}
```

Output:

1.

```
Generated Number: 96
Square of 96 = 9216

Generated Number: 90
Square of 90 = 8100

Generated Number: 53
Cube of 53 = 148877

Generated Number: 32
Square of 32 = 1024

Generated Number: 14
Square of 14 = 196

Generated Number: 93
Cube of 93 = 804357

Generated Number: 89
Cube of 89 = 704969

Process finished with exit code 130 (interrupted by signal 2:SIGINT)
```

2.

```
/opt/homebrew/Cellar/openjdk/23.0.1/libexec/openjdk.jdk/Contents/Home/bin/java -javaage
Generated Numbers: [76, 43, -71, 35, -59, -85, 44, -12, -34, 63, -1, 27, -70, 75, 0]
Positive Even: Positive Odd: Negative: 43 35 63 27 75
76 44
-71 -59 -85 -12 -34 -1 -70

Process finished with exit code 0
```

3.

```
/opt/homebrew/Cellar/openjdk/23.0.1/libexec/openjdk.jdk/Contents/Home/bin/java
Produced: 1
Consumed: 1
Produced: 2
Consumed: 2
Produced: 3
Consumed: 3
Produced: 4
Consumed: 4
Produced: 5
Consumed: 5
```

Post Lab Subjective/Objective type Questions:

1. What does join() method?

-> join() is used to make one thread wait for another to finish execution.

2. Is it possible to start a thread twice?

-> Once a thread has been started and finishes execution, you cannot do it again.

3. Can we call the run() method instead of start()?

•Calling run() directly just executes the method in the same thread (like a normal function call).

•Calling start() actually creates a new thread, which then executes run() asynchronously.

4. What is difference between wait() and sleep()?

5. What is the purpose of the Synchronized block?

-> The synchronized block ensures that only one thread at a time executes a specific section of code on the same object.

6. What is the difference between notify() and notifyAll()?

notify() -

Wakes up one random thread waiting on that object's monitor.

notifyAll() -

Wakes up all threads waiting on that monitor. They compete for the lock.

Conclusion:

All these concepts together define how threads coordinate, communicate, and safely share resources in Java.

SOMAIYA VIDYAVIHAR UNIVERSITY
K J Somaiya School of Engineering
(formerly K J Somaiya College of Engineering)

K. J. Somaiya School of Engineering, Mumbai-77

(Formerly K. J. Somaiya College of Engineering)

Department of Electronics Engineering

Somaiya TRUST

Signature of faculty in-charge with Date:

Roll No:_____